- Imperative commands (single line commands to create the objects)
  - **kubectl run my-pod --image=nginx** [here ***run*** is used, not ***create***]
    - **restartPolicy**: always

      always: restart pod even if the container exits with exit coode ***0***

      onFailure: restart pod only if container's exit code is non-zero

      never: never restart the pod even if container stops.
    - **namespace**: default

      it is *logical* grouping of a cluster to separate logically like dev, prod, ..etc.

      You can set the resource allocation according to namespace as well.
    - **labels**: ***run****: my-pod*
    - port: not exposed

      [port is never exposed from container though, means from Service you can route to any port of the pod you want; *port* only there for ease of quering pods according to port or to display the usable port of the contaienr to user when they get *kubectl descripe pod <pod name>*]

      ```yaml
      apiVersion: v1
      kind: Pod
      metadata:
        labels:
          run: my-pod
        name: my-pod
      spec:
        containers:
        - image: nginx
          name: my-pod
          resources: {}
        dnsPolicy: ClusterFirst
        restartPolicy: Always
      status: {}
      ```

      (default configs of the command)
  - **kubectl create deployment my-deploy --image=nginx** [here ***create*** is used]
    - Deployment name: my-deploy
    - ReplicaSet: my-deploy-<hash>
    - Pod: my-deploy-<hash>-<random>
    - labels: ***app****:my-deploy*

- These are the things kubectl create commands can create. pod cannot be created using kubectl create command

```
clusterrole             Create a cluster role
clusterrolebinding      Create a cluster role binding for a particular cluster role
configmap               Create a config map from a local file, directory or literal value
cronjob                 Create a cron job with the specified name
deployment              Create a deployment with the specified name
ingress                 Create an ingress with the specified name
job                     Create a job with the specified name
namespace               Create a namespace with the specified name
poddisruptionbudget     Create a pod disruption budget with the specified name
priorityclass           Create a priority class with the specified name
quota                   Create a quota with the specified name
role                    Create a role with single rule
rolebinding             Create a role binding for a particular role or cluster role
secret                  Create a secret using a specified subcommand
service                 Create a service using a specified subcommand
serviceaccount          Create a service account with the specified name
token                   Request a service account token
```

-
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-deployment
  name: my-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-deployment
  strategy: {}
  template:
    metadata:
      labels:
        app: my-deployment
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}
status: {}
```
(default configs of the command)

labels are automatically added.

- ***ReplicaSet*** cannot be created directly using imperative way, because Deployment is there to create and manage replicasets.
- ***kubectl expose*** takes replication controller, service, deployment, or pod and creates a *Service* to expose that.

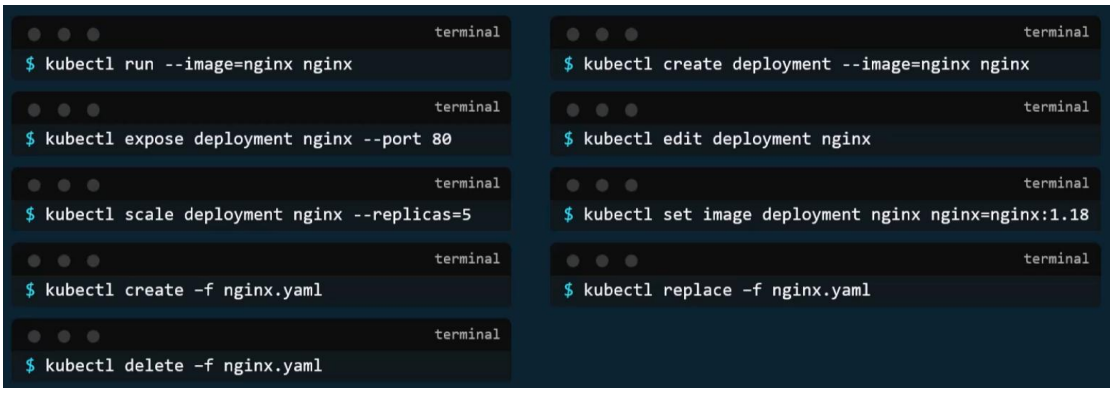
```
vagrant@controlplane:~$ kubectl get svc,pod
NAME                 TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP   36m

NAME          READY   STATUS    RESTARTS   AGE
pod/my-pod    1/1     Running   0          34s
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                 TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
service/kubernetes   ClusterIP   10.96.0.1      <none>        443/TCP   37m
service/my-pod       ClusterIP   10.96.68.232   <none>        80/TCP    6s

NAME          READY   STATUS    RESTARTS   AGE
pod/my-pod    1/1     Running   0          76s
```

- you can see here, It created a service named *my-pod*. Ifs of type ***ClusterIP*** which is default.
- If you give ***--type=NodePort*** flag, then it'll create a service of type NodePort

```
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80 --type NodePort
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                 TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
service/kubernetes   ClusterIP   10.96.0.1       <none>        443/TCP        39m
service/my-pod       NodePort    10.96.119.232   <none>        80:32530/TCP   3s
```

- 

[these are the imperative commands]

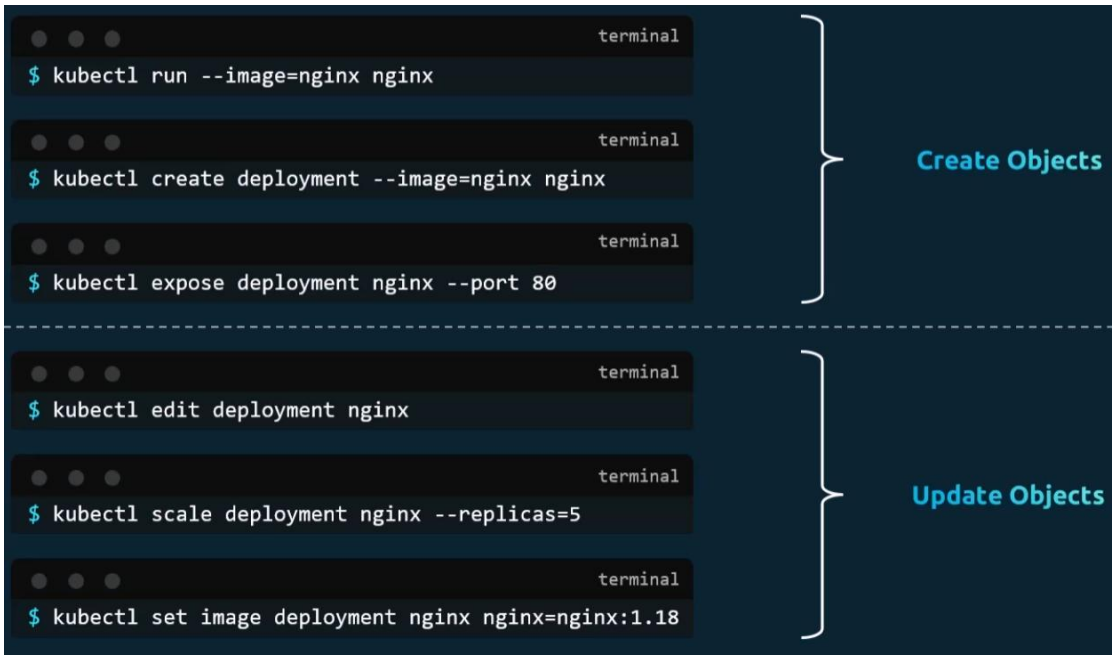
create -f is imperative because if already the same config present it'll fail, where as apply -f will not fail.

- Important trick
  - **kubectl create deployment my-deploy --image=nginx --dry-run=client -o yaml**

    kubectl run my-pod --image=nginx --dry-run=client -o yaml
    - it'll give the yaml file of default deployment, remove the unnecessary fields, customize that, and create deployment

- Because of the flag ***--dry-run=client*** it'll not create the deployment, just simulate and give the output as yaml.

- **kubectl get pod my-pod -o yaml**
  - It'll give the running config of the pod as an yaml.
  - Can be done with any types of objects i.e. pod, replica set, container runtime, deployment ..etc
  - But it'll contain a lot of things, that needs to be removed.



- **edit** vs **replace** command
  - *kubectl edit* command open the running status of the object (pod, replicaset, replication controller, deployment, service ..etc) in a yaml file.

    This configurations are there in the memory, not stored in the yaml file that was being used to create the object.

    So, if you do ***kubectl edit*** then it'll ont have any track of the changes. If someone later want to update the object, and he sees the yaml file present (not the running configuration yaml) and try to update, it might do some wrong changes in the running object.
  - *kubectl replace* command uses the physical yaml file to update the running object.

    It does ***dumb*** update in the running object. Whatever there in the physical yaml, it'll update all the things and delete the configs which are not present in that physical yaml file.

    By default it doesn't delete the current object (if anything doesn't match with physical yaml), to replace force fully you can give the flag ***--force***.

```
vagrant@controlplane:~$ kubectl replace --force -f pd.yaml
pod "my-pod" deleted from default namespace
pod/my-pod replaced
```

It looks similar to ***kubectl apply***, but *kubectl apply* does ***intelligent*** update. It finds the *diff* between running object and the physical yaml (used to update the object) and update those only.

It doesn't delete any configs which are not present in the current yaml but there in the running object.

kubectl apply is *declarative* command.

- If there are multiple files out of which objects will have to be created, then just give the path of the directory where those are present to *kubectl apply*, it'll create all those.

  kubectl apply -f <path where all files are present>

--- kubectl apply internals ---

- [here I'll take an example of a deployment named *my-dep*]
- There are 3 things.

  yaml (file that you write)

  last applied config (when you execute *kubectl* ***get*** *deploy my-dep* ***-o yaml***) **[JSON format]**

  live config (when you execute *kubectl* ***describe*** *deploy my-dep*)

  [*deploy* is shorthand of *deployment*]

  - Whenever you do *declarative* change using a yaml i.e.

    kubectl apply -f my-dep.yaml

    it changes both *live config*, *last applied config*. and obviously *yaml* is changed by you only.

  - Whenever you do *imperative* change using the command i.e.

    kubectl scale deployment my-dep --replicas=5

    it changes only *live config*. *Last applied config* will not be changed. And of cource, *yaml* has not been changed.

command type	yaml	last appplied config	live config
declarative	Y	Y	Y
imperative	N	N	Y

- There are a few commands [deployment is used for example; can be used for any objects]:

  - **kubectl diff -f <yaml file name>**

    compare the yaml with the *live config* along with last applied and display how it'll look like after you execute *kubectl apply*.

  - **kubectl apply view-last-updated deployment <deployment name>**

    show the last updated configs [which is present actually in JSON] in *yaml* format cleanly.

  - **kubectl get deployment <deployment name> -o yaml**

    kubectl get -f <deployment *yaml file* name> -o yaml [it fetch from the file]

    Both will display the live configs of the deployment object.

    Also, there will be a JSON structure under *annotations* field where you can see *last updated config*.

- Stage-1: [create a deployment from yaml]
  - Initial yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

  - **kubectl diff -f <**yaml file name**>** will display like below

```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-d
--- /tmp/LIVE-3981462885/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1106384729/apps.v1.Deployment.default.ng-dep
@@ -0,0 +1,41 @@
+apiVersion: apps/v1
+kind: Deployment
+metadata:
+  creationTimestamp: "2026-04-30T21:32:08Z"
+  generation: 1
+  name: ng-dep
+  namespace: default
```

[cropped]

  - Run the deployment using: *kubectl apply -f <yaml file name>*
  - **kubectl apply view-last-applied deployment <**deployment name**>**

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

- Stage 2: [change image using imperative command]
  - **kubectl set image deploy <deployment name> <container name>=<new image>**

    kubectl set image deploy ng-dep nginx=nginx:1.14.2

```
vagrant@controlplane:~$ kubectl set image deploy ng-dep nginx=nginx:1.14.2
deployment.apps/ng-dep image updated
```

  - **kubectl diff -f <**deployment file name**>**

```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-d
--- /tmp/LIVE-2986659376/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-1145231250/apps.v1.Deployment.default.ng-dep
@@ -6,7 +6,7 @@
     kubectl.kubernetes.io/last-applied-configuration: |
       {"apiVersion":"apps/v1","kind":"Deployment","metadata":{
ntainers":[{"image":"nginx:1.16.1","name":"nginx","ports":[{"co
   creationTimestamp: "2026-04-30T21:32:53Z"
-  generation: 2
+  generation: 3
   name: ng-dep
   namespace: default
   resourceVersion: "33813"
@@ -29,7 +29,7 @@
         app: nginx
     spec:
       containers:
-      - image: nginx:1.14.2
+      - image: nginx:1.16.1
         imagePullPolicy: IfNotPresent
```

    [you can see, yaml file has *nginx:1.16.1* but live config has *nginx:1.14.2*]

    [so, its showing `-` on *nginx:1.14.2* and `+` on *nginx:1.16.1*]

  - **kubectl apply view-last-applied deploy <**deployment name**>**

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

    [its showing image as *nginx:1.16.1* because imperative change doesn't effect last applied config]

- Stage-3: [update yaml, remove replica field, then declarative update]
  -
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```
  -
```
vagrant@controlplane:~$ kubectl diff -f simple_deployment.yaml
diff -u -N /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-
--- /tmp/LIVE-3508972230/apps.v1.Deployment.default.ng-dep
+++ /tmp/MERGED-2827279498/apps.v1.Deployment.default.ng-dep
@@ -6,14 +6,14 @@
     kubectl.kubernetes.io/last-applied-configuration: |
       {"apiVersion":"apps/v1","kind":"Deployment","metadata":
ntainers":[{"image":"nginx:1.16.1","name":"nginx","ports":[{"c
   creationTimestamp: "2026-04-30T21:32:53Z"
-  generation: 2
+  generation: 3
   name: ng-dep
   namespace: default
   resourceVersion: "33813"
   uid: 92189358-0dd5-4f8f-8144-1936af5322ab
 spec:
   progressDeadlineSeconds: 600
-  replicas: 3
+  replicas: 1
   revisionHistoryLimit: 10
   selector:
     matchLabels:
@@ -29,7 +29,7 @@
         app: nginx
     spec:
       containers:
-      - image: nginx:1.14.2
+      - image: nginx:1.16.1
```
  - [its showing if I update, *replica* and *image* will be updated]
  - Now in last applied config, *replicas* field would have been removed because we deleted that field in yaml.

```
vagrant@controlplane:~$ kubectl apply view-last-applied -f simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[replicas is not there, image is *nginx:1.16.1*]

- *live config*

```
spec:
  progressDeadlineSeconds: 600
  replicas: 1
```

```
spec:
  containers:
  - image: nginx:1.16.1
```

[the image and replicas got changed]

- Its looking like, why there is need of *last applied config* because when we apply, the live config is becoming same as the yaml.
- <u>Stage-1: [create declarative from yaml]</u>
  - See the yaml now

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.16.1
        ports:
        - containerPort: 80
```

[added one label only]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
  namespace: default
spec:
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - image: nginx:1.16.1
        name: nginx
        ports:
        - containerPort: 80
```

[last applied config will be same as yaml because of declarative change]

- Stage-2: [add one more label in imperative way; *kubectl edit* command]

```
labels:
  app: ng-deployment
  keyx: valx
```

[live config changed]

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
```

[last applied config will be same because it was imperative update]

- Stage-3: [update the replicas using declarative]

```
vagrant@controlplane:~$ cat simple_deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ng-dep
  labels:
    app: ng-deployment
spec:
  replicas: 3
  selector:
```

[updated yaml file]

```
    creationTimestamp: "2026
-   generation: 1
+   generation: 2
    labels:
      app: ng-deployment
      keyx: valx
@@ -16,7 +16,7 @@
    uid: 5071163c-87e2-49db-
  spec:
    progressDeadlineSeconds:
-   replicas: 1
+   replicas: 3
    revisionHistoryLimit: 10
```

[ignore generation]

[kubectl diff -f <deployment file name>]

you can see, its not showing *keys: valx* label to be deleted when it is not there in our current yaml file.

- kubectl apply -f <deployment file name>

```
vagrant@controlplane:~$ kubectl apply view-last-applied deploy ng-dep
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations: {}
  labels:
    app: ng-deployment
  name: ng-dep
  namespace: default
spec:
  replicas: 3
```

[last applied config]

```
  creationTimestamp: "2026-04-
  generation: 2
  labels:
    app: ng-deployment
    keyx: valx
  name: ng-dep
  namespace: default
  resourceVersion: "37619"
  uid: 5071163c-87e2-49db-a1a0
spec:
  progressDeadlineSeconds: 600
  replicas: 3
```

[live config, replicas got increased but that label (keyx: valx) is still there]

- So, how does ***kubectl apply*** work then?
  - It does the below

    Sets fields, that appear in the configuration (yaml) file, in the live configuration.

Clears fields, that removed from the configuration (yal) file, in the live configuration.

- It does the above after comparing with *last applied field*.
- If any field is there in [either of *yaml* or *last applied config*] (or) [the field is present in both *yaml* and *last applied config*]
  then that field will be updated in the *live config*.
- In the previous example. The label (keyx: valx) was not there in both yaml and *last applied config*, so it didn't track that.

> It tracks the configs which are present in either yaml or last applied config and updates the live config accordingly. If something is there in live config but not there in both yaml and last applied config then it'll skip that.

- ### Merging changes to primitive fields

  Primitive fields are replaced or cleared.

  Note: – is used for "not applicable" because the value is not used.

Field in object configuration file	Field in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Set live to configuration file value.
Yes	No	-	Set live to local configuration.
No	-	Yes	Clear from live configuration.
No	-	No	Do nothing. Keep live value.

  ### Merging changes to map fields

  Fields that represent maps are merged by comparing each of the subfields or elements of the map:

  Note: – is used for "not applicable" because the value is not used.

Key in object configuration file	Key in live object configuration	Field in last-applied-configuration	Action
Yes	Yes	-	Compare sub fields values.
Yes	No	-	Set live to local configuration.
No	-	Yes	Delete from live configuration.
No	-	No	Do nothing. Keep live value.

yaml	last appplied config	live config	RES
Y	Y	Y/N	yaml : Y last : update to yaml live : update to yaml
Y	N	Y/N	yaml : Y last : update to yaml live : update to yaml
N	Y	Y/N	yaml : N last : remove live : remove
N	N	Y	yaml : N last : N live : no update

- F

Namespaces

- Consider the below analogy

  - There are 2 houses,

    In one house (house-A), one person called Mark *Smith* is there.

    In another house (house-B), one person called Mark *Williams* is there.
  - In house-A, people can call *Mark Smith* as only *Mark* because there is no other guy having this name in that house.

    Similarly in house-B, people can call *Mark Williams* as *Mark* only.
  - But, if people of house-A wants to call *Mark Williams* then they need to call by full-name i.e. *Mark Williams*. Not only *Mark*.

    Similarly for people of house-B as well. They need to call *Mark Smith* .
  - A person who neither belong to house-A nor house-B, needs to call those by their full name only i.e. *Mark Smith* or *Mark Williams*.
  - Consider these houses as namespaces.

    Inside a namespace, resources having same names cannot be present (you can't create 2 pods having exactly same name). but, in multiple namespaces, resources of same name can be present.
  - For example, if you want multiple environments like *dev*, *prod* and you don't want to update any *production* pods while testing during *development* or something like that, ***namespaces*** help in that.

    You can have exact same resources in both the namespaces.

- Example:
  - Consider a single namespace scenario:
    - Some pods are running DB. And service name (ClusterIP) is ***db***.
    - The application pods who want to use database can use ***db*** directly to access the database.
  - Consider a multiple namespace scenario:

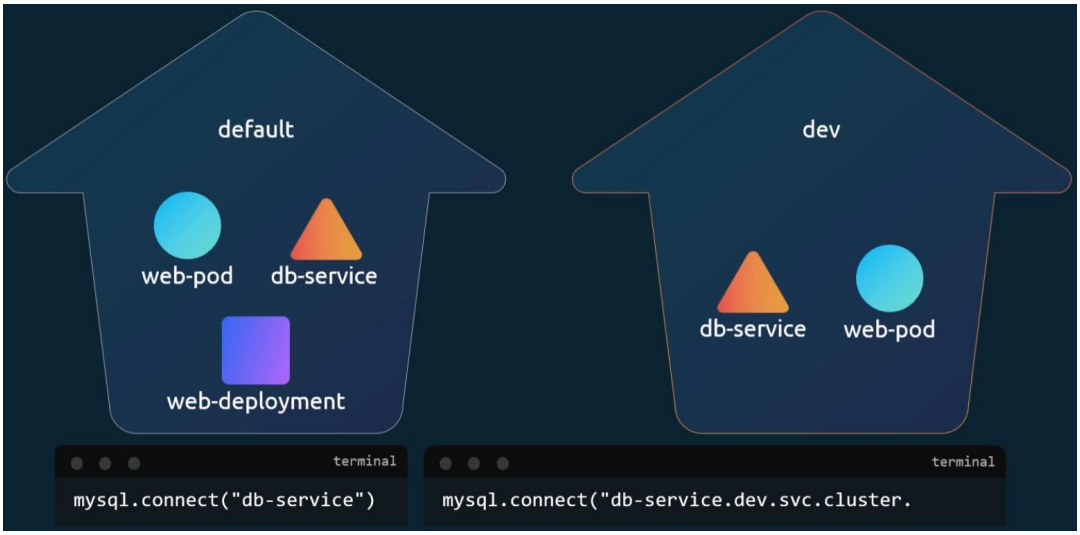
- In the same example as above, lets imagine 2 namespaces are there.
- Now, the application pods of one namespace can directly use ***db*** to access the database running in same namespace.

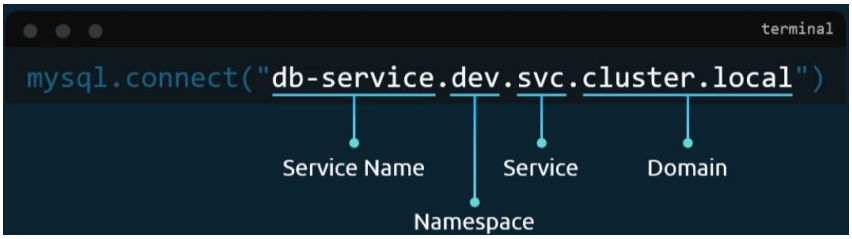
  But, if they want to use ***db*** of the other namespace then they need to give the full DNS path i.e. ***db.<namespace name>***

  or ***db.<namespace name>.svc.cluster.local*** [you don't need to give this, *db.<namespace name>* is converted to this internally]

- **db.<namespace name>.svc.cluster.local**

  in here, **svc** means don't access the pod directly, rather do using the ***db*** service. **cluster.local** is the internal DNS of the cluster. [**.local** is a top-level-domain just like *.com*, *.in* ..etc but it is internal]





- If you want to get or create the resources present in a specific namespace then

  kubectl get <resource type like *pod, rs, deploy***> -n <**namespace name**>**

  kubectl creae -f <yaml file name**> -n <namespace name>**

  [**NOTE:** You need to write '***create***', not '***apply***' when you are creating something from tha yaml first time (still have to verify this)]

  [if the **namespace** key is present in the *yaml* file already, then no need to give ***--namespace*** flag]

- Or, in ***yaml*** you can write ***namespace*** inside metadata section.

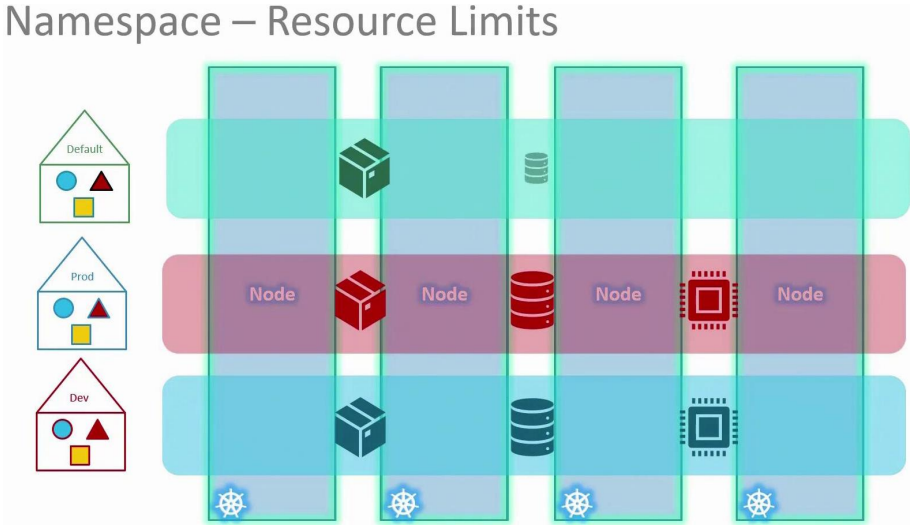
```yaml
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  namespace: dev
```

- To create a *namespace*, the yaml file format will be like this

```
vagrant@controlplane:~$ kubectl create namespace test --dry-run=client -o yaml
apiVersion: v1
kind: Namespace
metadata:
  name: test
```

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev
```

- All the ***namespace*** configures while setup:
  - When a kubernetes *cluster* is first setup, ***default*** namespace is created. Whatever resources you create, it is created inside this.
  - Kubernetes creates a set of Pods and Services for its internal puspose. These all stays in the namespace called ***kube-system*** .
  - There is another namespace called ***kube-public*** where resources available to all users are created.
- You can create **quota** (long format: **ResourceQuota**) to the namespaces (setting the limits)

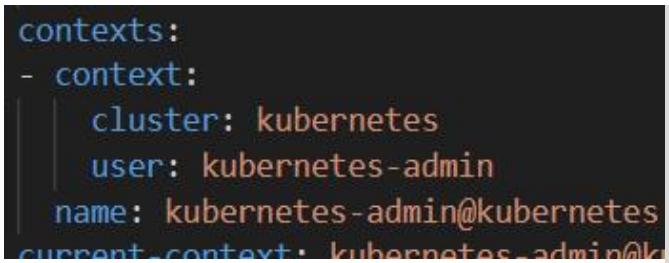


- If you want to change the default (not talking about the namespace having name *default*) namespace

  kubectl config set-context $(kubectl config current-context) --namespace=<namespae name which has to be made default**>**

- config -

- The config file path is ***~/.kube/config*** .
- 

- 

[if namespace is not there]

it looks something like this (its content format is of yaml, but the filename is ***config*** only, not ~~***config.yaml***~~)

- It mainly contains three things: **cluster, contexts, users**
- **clusters**
  - its an list of clusters.
  - Each cluster contains *name, cluster* (cluster contains *certificate-authority-data, server)* fields.
  - **name**
    - its just an identifier that will be used later inside *context* (you can see inside *context* field).
    - every operation i.e. create pod, get svc ..etc hits this URL.

      kubectl → API Server → Cluster

- ***cluster*.server**

  it's the end-point of ***API Server*** where all ***kubectl*** command goes.

- ***cluster*.certificate-authority-data**
  - Ensures you're talking to the real cluster
  - Prevents man-in-the-middle attacks

➢ **users**

- It contains the list of users.
- It defines "Who are you when talking to the Cluster"
- Each entries of *user* contain the below fields **name, user** (*user* contains *client-certificate-data, client-key-data*) fields.
- **name**

  just a identifier to the user. Which is referenced inside *context* field (you can see in the yaml).

- **user.client-certificate-data** & **user.client-key-data** (common in local setup)

  certificate is like ID card*;* ***key*** is like signature (Analogy)

- In real-world, instead of certificate and key, ***token*** is used.

  ```
  token: eyJhbGciOiJSUzI1NiIsIn
  ```

  - it is used in ServiceAccounts, CI/CD, API access.
- In cloud provider cases, external Auth is present.
- All-In-One, ***user*** field is for authentication with the ***cluster*** .

➢ **contexts**

- It is something that reference all the things i.e.

  context* = *cluster* + *user* + *default namespace **--- IMPORTANT ---**

- When you switch to a context, all the things will be switched i.e. user, cluster, *default* namespace
- When you execute any command like

  kubectl get pod

  it is executed like the below

  kubectl get pod --namespace=<default namespace name>

- If the namespace field is not present in the *config* file (refer the image), then it'll take the namespace called "**default**" by default.
- But, whatever namespace you give (override) like the below

  kubectl get pod --namespace=<namespace name other than context default one>

  the **user** and **cluster** will be same only.

➢ **NOTE** **namespace** field will not be there inside the ***config*** file. -----

- You can create as many context you want.
  - Same cluster + different namespaces : new context

    (**optional**, if you want to maintain different contexts; otherwise you can just override the default namespace and move on in the same context using *--namespace*=<other namespace name>
  - Same cluster + different user : new context

    (its **mandatory**)

    you cannot switch user independently, only you can switch *context*.
  - Different cluster : new context

    (its **mandatory**)
- **kubectl config --help**

```
Available Commands:
  current-context   Display the current-context
  delete-cluster    Delete the specified cluster from the kubeconfig
  delete-context    Delete the specified context from the kubeconfig
  delete-user       Delete the specified user from the kubeconfig
  get-clusters      Display clusters defined in the kubeconfig
  get-contexts      Describe one or many contexts
  get-users         Display users defined in the kubeconfig
  rename-context    Rename a context from the kubeconfig file
  set               Set an individual value in a kubeconfig file
  set-cluster       Set a cluster entry in kubeconfig
  set-context       Set a context entry in kubeconfig
  set-credentials   Set a user entry in kubeconfig
  unset             Unset an individual value in a kubeconfig file
  use-context       Set the current-context in a kubeconfig file
  view              Display merged kubeconfig settings or a specified kubeconfig file
```

  - **use-context** is used to set a new **current-context** (see in yaml).
- If you want to create a new context

  kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test

```
vagrant@controlplane:~$ kubectl config set-context test-context --cluster=kubernetes --user=kubernetes-admin --namespace=test
Context "test-context" created.
vagrant@controlplane:~$ kubectl config get-contexts
CURRENT   NAME                          CLUSTER      AUTHINFO           NAMESPACE
*         kubernetes-admin@kubernetes   kubernetes   kubernetes-admin
          test-context                  kubernetes   kubernetes-admin   test
```

- You can use this command to view all the configs.

  kubectl config view

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.56.11:6443
  name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: kubernetes-admin
  name: kubernetes-admin@kubernetes
current-context: kubernetes-admin@kubernetes
```

[cropped, it'll display the whole config file]

- If you want to change the ***current-context*** then

  kubectl config use-context <context name**>**

- V

---------- THE END OF *CONFIG* ----------

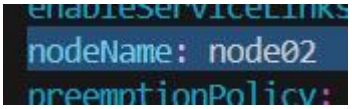
- There is another type of resource called **ResourceQuota** which is used to set the limit of a specific namespace.

```yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: compute-quota
  namespace: dev
spec:
  hard:
    pods: "10"
    requests.cpu: "4"
    requests.memory: 5Gi
    limits.cpu: "10"
    limits.memory: 10Gi
```

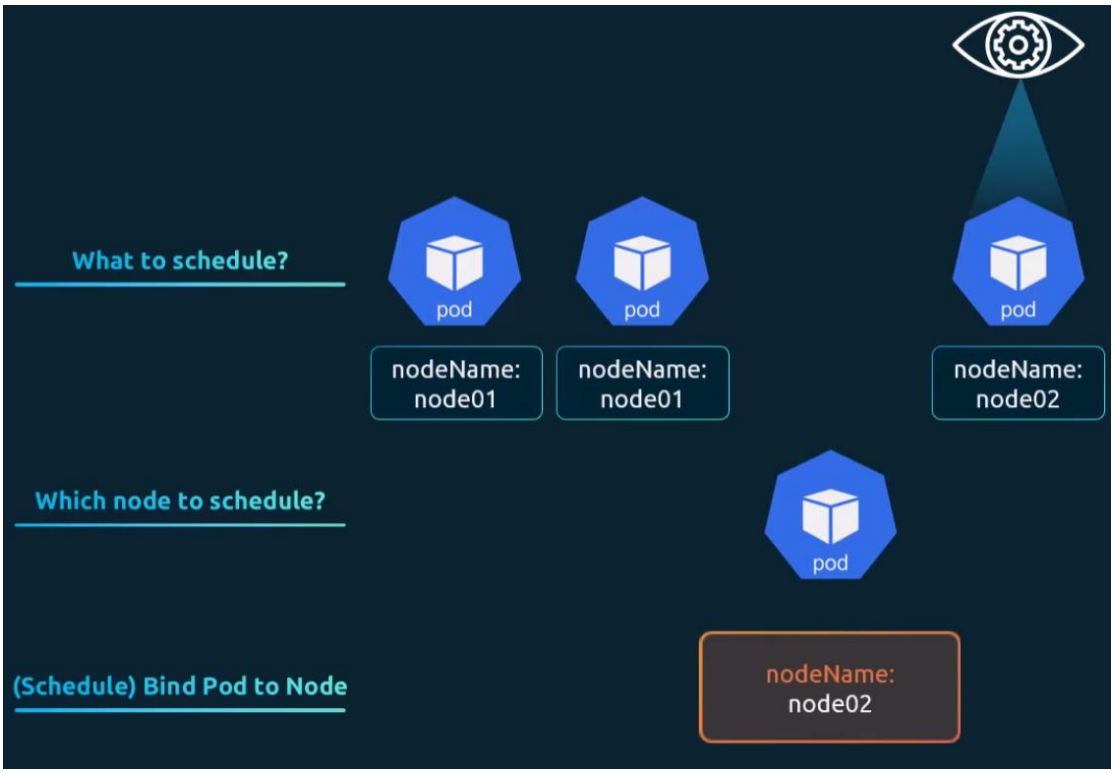
-

- Scheduling -

- In a running pod, when you see it, there will be a field called **nodeName** will be present, which contains the worker node's name where that specific pod is running.
  Just execute *kubectl edit pod* <pod name> to see this.



- How ***scheduling*** works?



  - When you run a pod (without specifying *nodeName* field), the pod object is created.
  - Then, *scheduler* sees this, and assign a node to it (***nodeName***).
    NOTE: this ***nodeName*** field is under ***spec*** field of *Pod's* config.
  - Then, one **binding** will be created *in the target node* which will bind the *pod* to the *target node*.

- If the Scheduler doesn't exist,
  - then *nodeName* will not be appended to the pod's config.
    Even if you give *nodeName*, ***binding*** object will not be created in the target node unless manually send the POST request to the target worker node to create Binding object.

- In this case, the Pod will be in pending state.

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node03
```

```
Every 2.0s: kubectl get pod

NAME                      READY   STATUS    RESTARTS   AGE
ng-dep-66b9cb6d4d-4r822   1/1     Running   0          38h
ng-dep-66b9cb6d4d-f4gl4   1/1     Running   0          38h
ng-dep-66b9cb6d4d-f52lt   1/1     Running   0          39h
test-pod                  0/1     Pending   0          13s
```

  - I simulated is giving a *nodeName* which doesn't exist inside */etc/hosts* file. So, the *Binding* object will not be created.
- After sometime, the pod will be **deleted** because its status couldn't be 'Running'.

➢ Simulating **Manual Scheduling** of a Pod to a Node

- First create a yaml config of the node giving ***nodeName*** field.

```yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: test-pod
  name: test-pod
spec:
  containers:
  - image: nginx
    name: test-pod
  nodeName: node01
```

- Now, create the pod

```
vagrant@controlplane:~$ kubectl create -f test-pod.yaml
pod/test-pod created
```

- Now, you need to create a *Binding* yaml file.

```yaml
apiVersion: v1
kind: Binding
metadata:
  name: test-binding
target:
  apiVersion: v1
  kind: Node
  name: node01
```

NOTE: It doesn't contain any data about the Pod, it only contains the details of the target node.

- The yaml is just to understand the key value pairs, in real there is **no such long-lived object called *Binding* exists**.

  Its just the JSON payload to be sent in the POST request to bind the pod to the target node.

- Now, you need to trigger the POST request to the **API Server** to bind the pod to the *target worker node*.

  NOTE: the worker node doesn't expose any end-points, only API Server in the master node does this.

  And, you need to trigger the request to this API Server's end point.

- The POST request looks like this (just an example)

```
curl --header "Content-Type: application/json" \
     --request POST \
     --data '{"apiVersion":"v1", "kind": "Binding", "metadata": {"n
     http://$SERVER/api/v1/namespaces/default/pods/$PODNAME/binding
```

  - You'll get the $SERVER from ***kubectl config view*** command. It'll be in the form

    <API server node's IP**>:<**API Server's port**>**

```
vagrant@controlplane:~$ kubectl config view
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://192.168.56.11:6443
```

- $PODNAME is the Pod's name that you get from ***kubectl get pod*** command.

```
vagrant@controlplane:~$ kubectl get pod
NAME                      READY   STATU
ng-dep-66b9cb6d4d-4r822   1/1     Runni
ng-dep-66b9cb6d4d-f4gl4   1/1     Runni
ng-dep-66b9cb6d4d-f52lt   1/1     Runni
test-pod                  1/1     Runni
```

- Now, trigger the request

```
curl --header "Content-Type: application/json" \
 --request POST \
 --data '{"apiVersion":"v1","kind":"Binding","metadata":{"name":"test-binding"},"target":{"apiVersion":"v1","kind":"Node","name":"node01"}}' \
https://192.168.56.11:6443/api/v1/namespaces/default/pods/test-pod/binding/
```

➢

- Labels and Selectors -

-
- F
- F
- F
- F
- F
- f
- F

Important Points

- Try to use *imperative commands* as much as you can in Exam.
- If you forget the keywords and resources, then execute ***kubectl api-resources*** you'll get the keywords.

```
vagrant@controlplane:~$ kubectl api-resources
NAME                              SHORTNAMES   APIVERSION                        NAMESPACED   KIND
bindings                                       v1                                true         Binding
componentstatuses                 cs           v1                                false        ComponentStatus
configmaps                        cm           v1                                true         ConfigMap
endpoints                         ep           v1                                true         Endpoints
events                            ev           v1                                true         Event
limitranges                       limits       v1                                true         LimitRange
namespaces                        ns           v1                                false        Namespace
nodes                             no           v1                                false        Node
persistentvolumeclaims            pvc          v1                                true         PersistentVolumeClaim
persistentvolumes                 pv           v1                                false        PersistentVolume
pods                              po           v1                                true         Pod
podtemplates                                   v1                                true         PodTemplate
replicationcontrollers            rc           v1                                true         ReplicationController
resourcequotas                    quota        v1                                true         ResourceQuota
secrets                                        v1                                true         Secret
serviceaccounts                   sa           v1                                true         ServiceAccount
services                          svc          v1                                true         Service
mutatingadmissionpolicies                      admissionregistration.k8s.io/v1   false        MutatingAdmissionPolicy
mutatingadmissionpolicybindings                admissionregistration.k8s.io/v1   false        MutatingAdmissionPolicyBinding
mutatingwebhookconfigurations                  admissionregistration.k8s.io/v1   false        MutatingWebhookConfiguration
```

[you can see there is a column 'KIND' which you can directly write in the yaml]

- Shortnames are also there to use in *imperative commands* which will SAVE TIME.

- If you want to get a documentation regarding any specific resource, you can use ***kubectl explain <rource name>*** command.

```
vagrant@controlplane:~$ kubectl explain pods
KIND:       Pod
VERSION:    v1

DESCRIPTION:
    Pod is a collection of containers that can run on a host. This resource is
    created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
    APIVersion defines the versioned schema of this representation of an object.
    Servers should convert recognized schemas to the latest internal value, and
    may reject unrecognized values. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources

  kind  <string>
    Kind is a string value representing the REST resource this object
    represents. Servers may infer this from the endpoint the client submits
    requests to. Cannot be updated. In CamelCase. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds

  metadata      <ObjectMeta>
    Standard object's metadata. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata

  spec  <PodSpec>
    Specification of the desired behavior of the pod. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status

  status        <PodStatus>
    Most recently observed status of the pod. This data may not be up to date.
    Populated by the system. Read-only. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status
```

- To go deeper into the specific fields of that resources,

  kubectl explain <resource name>.<field name>

  [you can nest these any number times:

  resource.field.sub_field.sub_sub_field …. etc]

```
vagrant@controlplane:~$ kubectl explain pods.spec
KIND:       Pod
VERSION:    v1

FIELD: spec <PodSpec>


DESCRIPTION:
    Specification of the desired behavior of the pod. More info:
    https://git.k8s.io/community/contributors/devel/sig-architecture/api-convent
    PodSpec is a description of a pod.

FIELDS:
  activeDeadlineSeconds <integer>
    Optional duration in seconds the pod may be active on the node relative to
```

  If you want, more comprehensive, then use

  kubectl explain <resource name> --recursive

```
vagrant@controlplane:~$ kubectl explain pods --recursive
KIND:       Pod
VERSION:    v1

DESCRIPTION:
    Pod is a collection of containers that can run on a hos
    created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind  <string>
  metadata      <ObjectMeta>
    annotations <map[string]string>
    creationTimestamp   <string>
    deletionGracePeriodSeconds  <integer>
```

- To get a basic yaml file instead of writing from scratch, use ***--dry-run=client -o yaml*** with the command.

  [with *run, create* for static] [with *get* if the object is already running]

  kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

  kubectl run my-pod --image nginx --dry-run client -o yaml

```
vagrant@controlplane:~$ kubectl run my-pod --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
  name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

```
vagrant@controlplane:~$ kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-dep
  name: my-dep
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-dep
  strategy: {}
  template:
    metadata:
      labels:
        app: my-dep
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}
```

- ۴
-
- F
- F
- F
- F